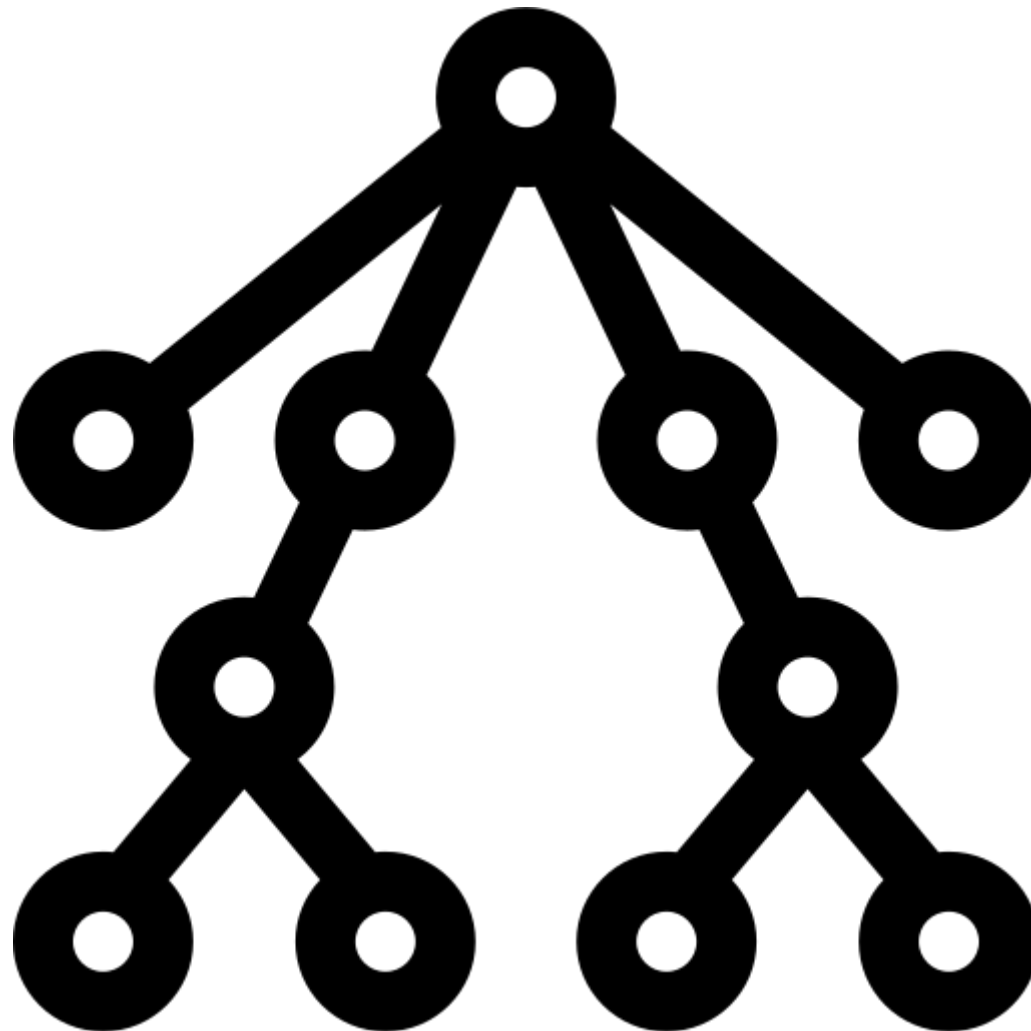
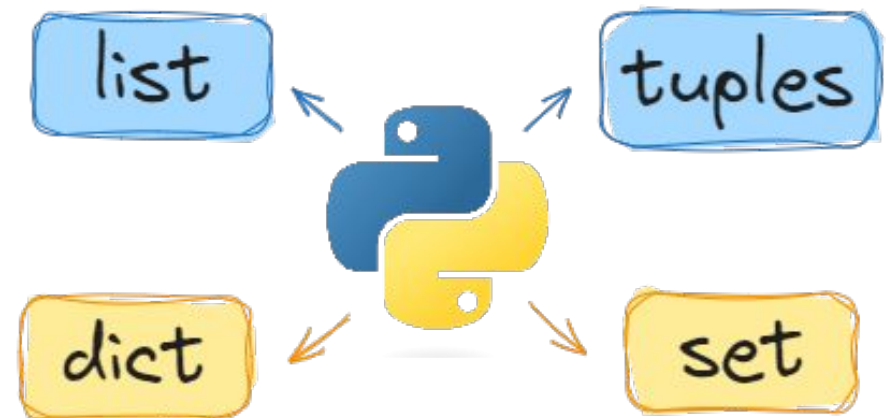


Estruturas de Dados e Funções em Python



Estruturas de Dados em Python

- Em Python, estruturas de dados são componentes fundamentais para armazenar e manipular coleções de dados. As principais estruturas de dados em Python são:
 - Listas (List) [];
 - Tuplas (Tuple) ();
 - Conjuntos (Set) { };
 - Dicionários (Dict) { : }.



Estruturas de Dados em Python -

Listas (List) []

- As listas são coleções **ordenadas**, **mutáveis** e permitem **valores duplicados**. Para criar uma lista, basta declarar uma variável e inicializá-la com os elementos entre colchetes []. Cada item dentro da lista deve ser separado por vírgulas.
- Você pode exibir diretamente o conteúdo de uma lista ou acessar elementos específicos usando índices.

```
frutas = ["Banana", "Laranja", "Maçã", "Manga"]  
print(frutas)  
["Banana", "Laranja", "Maçã", "Manga"]
```

Declarando uma lista de inteiros

```
meusNumeros = [1, 2, 3, 4]
```

Declarando uma lista de strings

```
contatinhos = ["José", "Vitor", "André", "Fabio"]
```

Declarando uma lista vazia

```
vazio = []
```

```
print(frutas[1])
```

```
Laranja
```

```
print(frutas[3])
```

```
Manga
```

```
print(frutas[4])
```

```
Undefined
```

Estruturas de Dados em Python -

Listas (List) []

- Acessamos os elementos de uma lista pelo índice. O índice das listas começa em 0. Python também permite usar índices negativos para acessar os elementos a partir do final da lista;
- Listas podem conter elementos de diferentes tipos ao mesmo tempo, o que as torna estruturas heterogêneas.

```
nomes = ["Ana", "Bruno", "Carlos"]
```

```
print(nomes[0]) # "Ana"  
print(nomes[1]) # "Bruno"  
print(nomes[2]) # "Carlos"  
print(nomes[-1]) # "Carlos"  
print(nomes[-2]) # "Bruno"
```



```
mistura = [10, "texto", 3.14, True, [1, 2, 3]]  
print(mistura)
```

Estruturas de Dados em Python -

Listas (List) []

- Principais operações:
 - **append()**: Adiciona um elemento no final da lista;
 - **remove()**: Remove a primeira ocorrência de um valor da lista;
 - **pop()**: Remove e retorna um elemento pelo índice ou, por padrão, remove o último;

Estruturas de Dados em Python -

Listas (List) []

- Principais operações:
 - **insert()**: Insere um elemento em uma posição específica;
 - **sort()**: Ordena a lista em ordem crescente (ou alfabética). Altera a lista original.
 - Para ordem decrescente, **reverse=True**;
 - **reverse()**: Inverte a ordem dos elementos sem ordenar;
 - **len()**: Retorna o tamanho da lista (quantidade de elementos).

Estruturas de Dados em Python -

Listas (List) []

```
frutas = ["maçã", "banana"]
frutas.append("uva")
print(frutas) # ['maçã', 'banana', 'uva']
```

```
frutas = ["maçã", "banana", "laranja"]
ultima = frutas.pop()
print(ultima) # 'laranja'
print(frutas) # ['maçã', 'banana']
```

```
frutas.pop(0) # Remove "maçã" índice 0
print(frutas) # ['banana']
```

```
frutas = ["banana", "laranja"]
frutas.insert(0, "maçã") # índice 0
print(frutas) # ['maçã', 'banana', 'laranja']
```

```
frutas = ["maçã", "banana", "laranja"]
frutas.remove("banana")
print(frutas) # ['maçã', 'laranja']
```

```
numeros = [3,1,2,4]
numeros.sort()
print(numeros) #[1, 2, 3, 4]
```

```
letras = ["c","a","b"]
letras.reverse()
print(letras) #['b', 'a', 'c']
```

```
frutas = ["maçã", "banana", "laranja"]
print(len(frutas)) # 3
```


Estruturas de Dados em Python -

Listas (List) []

```
tarefas = []

tarefas.append("Estudar Python")
tarefas.append("Fazer exercícios")
tarefas.append("Ler documentação")

print("Tarefas iniciais:")
print(tarefas)

tarefas.insert(0, "Tomar café")

tarefas.remove("Fazer exercícios")

print("\nTarefas após alterações:")
for i in range(len(tarefas)):
    print(f"{i + 1}. {tarefas[i]}")
```

```
tarefas.sort()
print("\nTarefas em ordem alfabética:")
print(tarefas)

tarefas.reverse()
print("\nTarefas em ordem reversa:")
print(tarefas)

ultima = tarefas.pop()
print(f"\nÚltima tarefa removida: {ultima}")

print(f"\nNúmero de tarefas restantes:
{len(tarefas)}")
```

```
nomes = ["Ana", "Bruno", "Carla"]
for nome in nomes:
    print("Olá", nome)
```


Estruturas de Dados em Python - Tuplas (Tuple) ()

- Tuplas são estruturas de dados muito parecidas com listas, mas com uma diferença importante: são imutáveis. Isso significa que, uma vez criada, você não pode modificar os elementos da tupla (não pode adicionar, remover ou alterar valores);
- São usadas para armazenar coleções de dados que devem permanecer imutáveis, garantindo a integridade dessas informações.

Estruturas de Dados em Python - Tuplas (Tuple) ()

- Elas são ideais para representar valores fixos, como coordenadas ou configurações, além de facilitar o retorno de múltiplos valores em funções.
- Por serem imutáveis, tuplas podem ser usadas como chaves em dicionários ou elementos em conjuntos, algo que listas não permitem. Além disso, tuplas oferecem melhor desempenho e menor consumo de memória em comparação às listas, tornando-as uma escolha eficiente para dados constantes.

Estruturas de Dados em Python - Tuplas (Tuple) ()

- Sintaxe básica: `minha_tupla = (1, 2, 3)`
- Você também pode criar uma tupla sem parênteses, apenas com vírgulas: `outra_tupla = 4, 5, 6`
- E uma tupla com um único elemento precisa de uma vírgula: `tupla_unitaria = (10,)`
- O acesso a elementos e a iteração com `for` ocorrem da mesma maneira que com listas:

```
cores = ("vermelho", "verde", "azul")  
print(cores[0])  # vermelho
```

```
for cor in cores:  
    print(cor)
```

Estruturas de Dados em Python - Tuplas (Tuple) ()

- Desempacotar tupla:

```
ponto = (3, 4)
x, y = ponto
print(x)  # 3
print(y)  # 4
```

- Lista de tuplas:

```
alunos = [("Ana", 8.5), ("Bruno", 7.0), ("Carla", 9.3)]

for nome, nota in alunos:
    print(f"{nome} tirou nota {nota}")
```

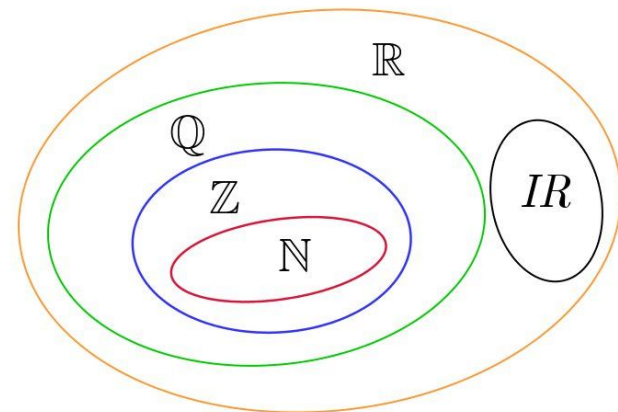
Estruturas de Dados em Python - Tuplas (Tuple) ()

```
produtos = [  
    (1001, "Caneta", 1.50),  
    (1002, "Caderno", 12.00),  
    (1003, "Mochila", 75.90),  
]  
  
codigo_busca = 1002  
  
for produto in produtos:  
    codigo, nome, preco = produto  
    if codigo == codigo_busca:  
        print(f"Produto encontrado: {nome} - R$ {preco:.2f}")  
        break  
else:  
    print("Produto não encontrado.")
```

Estruturas de Dados em Python - Conjuntos (Set) { }

- Conjuntos são coleções de elementos únicos (sem repetição). Eles são muito úteis quando você quer garantir que não haja duplicatas e para operações matemáticas como união, interseção e diferença.
- Como criar um conjunto:

```
meu_conjunto = {1, 2, 3, 4, 5}  
print(meu_conjunto)
```

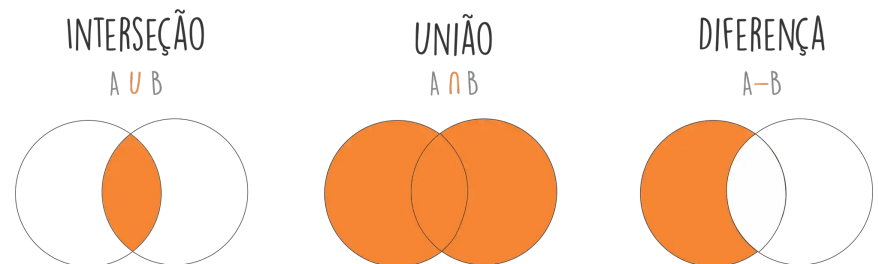


Estruturas de Dados em Python - Conjuntos (Set) { }

- Características importantes:
 - Elementos não podem se repetir. Exemplo:

```
conjunto = {1, 2, 2, 3}
print(conjunto)    # Saída: {1, 2, 3}
```

- Conjuntos são mutáveis, você pode adicionar ou remover elementos;
- Elementos precisam ser imutáveis (por exemplo, números, strings, tuplas), então listas não podem ser elementos de conjuntos.



Estruturas de Dados em Python - Conjuntos (Set) { }

- Adicionar e remover elementos:

```
conjunto = {1, 2, 3}
conjunto.add(4)      # adiciona 4
conjunto.discard(2)  # remove 2 (sem erro se não existir)
print(conjunto)      # {1, 3, 4}
```

- Operações comuns com conjuntos:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print("União:", A | B)      # {1, 2, 3, 4, 5, 6}
print("Interseção:", A & B)  # {3, 4}
print("Diferença A - B:", A - B) # {1, 2}
print("Diferença B - A:", B - A) # {5, 6}
```

Estruturas de Dados em Python - Conjuntos (Set) { }

```
curso_python = ["Ana", "Bruno", "Carla", "Daniel"]
curso_java = ["Bruno", "Elisa", "Carla", "Fernando"]

# Convertendo listas em conjuntos para operações de conjuntos
set_python = set(curso_python)
set_java = set(curso_java)

# Alunos que estão em ambos os cursos (interseção)
alunos_ambos = set_python & set_java

# Alunos que estão em pelo menos um dos cursos (união)
alunos_todos = set_python | set_java

# Alunos que estão apenas em Python (diferença)
apenas_python = set_python - set_java

# Alunos que estão apenas em Java (diferença)
apenas_java = set_java - set_python

print("Alunos em ambos os cursos:", alunos_ambos)
print("Alunos em pelo menos um dos cursos:", alunos_todos)
print("Alunos apenas no curso de Python:", apenas_python)
print("Alunos apenas no curso de Java:", apenas_java)
```

Estruturas de Dados em Python -

Dicionários (Dict) { : }

- Dicionários são estruturas de dados que armazenam pares de chave-valor. Cada chave é única e mapeia para um valor. São ótimos para representar dados que associam uma informação a outra, como um cadastro, onde a chave pode ser o nome e o valor o telefone, por exemplo.
- Criando um dicionário:

```
# Criando um dicionário com chaves e valores
pessoa = {
    "nome": "Ana",
    "idade": 25,
    "cidade": "São Paulo"
}

print(pessoa)
```

Estruturas de Dados em Python -

Dicionários (Dict) { : }

- Acessando valores:

```
print(pessoa["nome"])    # Ana
print(pessoa.get("idade")) # 25
```

- Adicionar ou modificar elementos:

```
pessoa["email"] = "ana@example.com" # adiciona novo par chave-valor
pessoa["idade"] = 26                # modifica o valor da chave 'idade'
```

- Remover elementos:

```
del pessoa["cidade"]          # remove a chave 'cidade'
email = pessoa.pop("email")    # remove e retorna o valor associado a 'email'
```

- Iterar sobre dicionários:

```
#pessoa = {'nome': 'Ana', 'idade': 26, 'cidade': 'São Paulo', 'email': 'ana@example.com'}
for chave, valor in pessoa.items():
    print(f"{chave}: {valor}")
```

Estruturas de Dados em Python -

Dicionários (Dict) { : }

```
contatos = {  
    "Ana": "9999-1111",  
    "Bruno": "8888-2222",  
    "Carla": "7777-3333"  
}  
  
print("Lista de contatos:")  
for nome, telefone in contatos.items():  
    print(f"{nome}: {telefone}")  
  
nome_novo = "Daniel"  
telefone_novo = "6666-4444"  
  
if nome_novo in contatos:  
    print(f"Contato {nome_novo} já existe. Atualizando telefone.")  
else:  
    print(f"\nAdicionando contato {nome_novo}.")  
  
contatos[nome_novo] = telefone_novo  
  
print(contatos)
```

Estruturas de Dados em Python -

Dicionários (Dict) { : }

```
nome_update = "Ana"
telefone_update = "9999-0000"

if nome_update in contatos:
    print(f"\nAtualizando telefone do contato {nome_update}.")
    contatos[nome_update] = telefone_update
else:
    print(f"\nContato {nome_update} não encontrado para atualizar.")

print("Contatos após adição e atualização:")
for nome, telefone in contatos.items():
    print(f"{nome}: {telefone}")
```

Estruturas de Dados em Python -

Dicionários (Dict) { : }

```
nome_remove = "Bruno"
if nome_remove in contatos:
    contatos.pop(nome_remove) # remove pela chave
    print(f"\nContato {nome_remove} removido.")
else:
    print(f"\nContato {nome_remove} não encontrado para remoção.")

nome_remove2 = "Elisa"
if nome_remove2 in contatos:
    contatos.pop(nome_remove2)
    print(f"\nContato {nome_remove2} removido.")
else:
    print(f"\nContato {nome_remove2} não encontrado para remoção.")

print("\nContatos finais:")
for nome, telefone in contatos.items():
    print(f"{nome}: {telefone}")
```


Estruturas de Dados em Python - Resumo

Estrutura	Mutável	Ordenável	Duplicados	Acessado por índice	Chave:Valor
<code>list []</code>	sim	sim	sim	sim	não
<code>tuple ()</code>	não	sim	sim	sim	não
<code>set {}</code>	sim	não	não	não	não
<code>dict {:}</code>	sim	sim	não (nas chaves)	não	sim

Funções em Python

Criar um programa que ajude a organizar uma lista de compras de supermercado.

1. Crie uma lista chamada `compras` que comece com 3 itens básicos (ex: `"Arroz"`, `"Feijão"`, `"Ovo"`).
2. Adicione dois novos itens à lista usando o método adequado (`.append()`).
3. Remova um item que você desistiu de comprar.
4. Exiba na tela quantos itens existem na lista atualmente usando a função `len()`.
5. Mostre a lista final ordenada alfabeticamente.

Criar um dicionário para armazenar informações sobre um produto de uma loja (nome, preço e estoque).

1. Crie um dicionário chamado `produto` com as seguintes chaves e valores iniciais:
 - `nome`: "Notebook"
 - `preco`: 2500.00
 - `quantidade`: 8
2. Imagine que o preço do notebook subiu. Atualize o valor da chave `preco` para `2700.00`.
3. A loja vendeu 2 notebooks. Atualize a chave `quantidade` diminuindo esse valor (deve passar a ser 6).
4. Adicione uma nova informação (uma nova chave) chamada `marca` com o valor `"TechBrand"`.
5. Exiba no terminal uma frase clara mostrando o nome do produto e o seu preço atualizado.

Funções em Python

- Funções são blocos de código que executam uma tarefa específica e podem ser reutilizadas várias vezes ao longo do programa. Elas ajudam a organizar o código, evitando repetição e tornando-o mais legível;
- Usamos a palavra-chave `def`, seguida do nome da função, parênteses (que podem ou não conter parâmetros) e dois pontos. O código da função vem indentado;

Funções em Python

- Para chamar uma função basta usar o nome da função seguido de parênteses e seus respectivos parâmetros, caso tenha.

```
#sem parâmetros
def minha_funcao():
    print("Olá, Esta é uma função simples!")

minha_funcao()
```

```
#com parâmetros
def saudacao(nome):
    print(f"Olá, {nome}! Seja bem-vindo(a)!")

saudacao("Ana")
saudacao("Maria")
```

Funções em Python

- Funções também podem retornar um resultado usando a palavra `return`:

```
def soma(a, b):  
    return a + b  
  
resultado = soma(3, 5)  
print("A soma é:", resultado)
```

```
def numeros_pares(lista_numeros):  
    pares = []  
    for num in lista_numeros:  
        if num % 2 == 0:  
            pares.append(num)  
    return pares  
  
#popular uma lista de forma automática  
numeros = []  
for y in range(30):  
    numeros.append(y)  
  
print(f"numeros: {numeros}")  
  
x= numeros_pares(numeros)  
print("Números pares:", x)
```

Funções em Python

- `return`:

```
def criar_pessoa(nome, idade, cidade):  
    pessoa = {  
        "nome": nome,  
        "idade": idade,  
        "maior_idade": idade >= 18, #teste booleano  
  
        "cidade": cidade  
    }  
    return pessoa  
  
dados = criar_pessoa("Ana", 25, "São Paulo")  
print(dados)
```

```
def coordenadas():  
    latitude = -23.4545  
    longitude = -47.7888  
    return [latitude, longitude]  
  
local = coordenadas()  
print(f"Latitude: {local[0]}\nLongitude: {local[1]}")
```

Funções em Python

List comprehension

- É uma forma reduzida de criar listas de forma rápida e prática.

Permite transformar e filtrar elementos e gerar listas.

Sintaxe:

```
[expressão for item in iterável if condição]
```

expressão: o que será colocado na nova lista.

iterável: elemento que possa ser percorrido.

condição: opcional - filtra os itens.

Funções em Python

List comprehension

Sintaxe

```
# variável = [expressão for item in iterável if condição]
```

Preencher uma lista com 20 numeros

```
numeros = [x for x in range(20)]
```

```
print("Lista gerada:")
```

```
print(numeros)
```

#Dobrar valores

```
numeros = [1,2,3,4]
```

```
dobrados = [x * 2 for x in numeros]
```

```
print("\nOriginal:")
```

```
print(numeros)
```

```
print("Dobrados:")
```

```
print(dobrados)
```

#Sem list comprehension

```
pares=[]
```

```
for i in range(30):
```

```
    if i % 2 == 0:
```

```
        pares.append(i)
```

```
print(pares)
```

#Com list comprehension

```
pares = [x for x in range(30) if x % 2 == 0]
```

```
print(pares)
```


Funções em Python

Função lambda

- `lambda` é uma forma reduzida de criar funções sem nome. Útil para funções simples, que geralmente são argumentos para outras funções.

Sintaxe:

```
lambda argumentos : expressão
```

argumentos: o que a função recebe.

expressão: a operação realizada sobre os argumentos.

Funções em Python

Função lambda

- `lambda` simples:

#Número positivo

```
positivo = lambda x: x > 0
print("Positivo:")
print(positivo(3))  # True
```

#Calcular o módulo

```
modulo = lambda x: x if x >= 0 else -x
print("Modulo:")
print(modulo(-5))  # 5
```

#Média simples

```
media = lambda a, b: (a + b) / 2
print("Media:")
print(media(10, 10))  # 10.0
```

#Par ou ímpar

```
tipo = lambda x: "par" if x % 2 == 0 else "ímpar"
print("Par ou Ímpar:")
print(tipo(3))  # ímpar
```

#Verificar se uma palavra começa com letra maiúscula

```
maiuscula = lambda s: s[0].isupper()
print("Começa maiuscula:")
print(maiuscula("Python"))  # True
```

#Substituir espaços por hífen

```
hifenizar = lambda s: s.replace(" ", "-")
print("Com hifen:")
print(hifenizar("Olá mundo"))  # Olá-mundo
```

Funções em Python

Funções: lambda, sorted().

- `sorted()` ordena elementos iteráveis. Retorna uma nova lista e não modifica o objeto original.

Sintaxe:

```
sorted(iteravel, key=None, reverse=False)
```

iterável: objeto a ser percorrido.

key: opcional - função, base para a ordenação.

reverse: opcional - se `True`, ordena em ordem reversa.

O parâmetro **key** deve ser usado quando a ordenação depender de elementos internos das estruturas da lista, como no caso de listas de dicionários.

Funções em Python

Funções: lambda, sorted().

- `sorted()` com `lambda`:

```
#Acessar item específico
```

```
dic_pessoa = {"nome": "Ana", "idade": 25}
```

```
pessoa = lambda x: x["idade"]
```

```
print(pessoa)
```

```
print(pessoa(dic_pessoa))
```

```
#Ordenar
```

```
lista_pessoa = [{"nome": "Ana", "idade": 25},  
                {"nome": "Maria", "idade": 2}]
```

```
lista_pessoa_ordenada = sorted(lista_pessoa, key=lambda x: x["idade"])
```

```
print("Original:")
```

```
print(lista_pessoa)
```

```
print("\nOrdenada")
```

```
print(lista_pessoa_ordenada)
```

Funções em Python

Funções: lambda, sorted().

- `sorted()` com `lambda`:

```
#Ordenar lista de dicionários
```

```
peessoas = [    {"nome": "Ana", "idade": 25},  
                {"nome": "Carlos", "idade": 30},  
                {"nome": "Bruno", "idade": 20}    ]
```

```
#Ordenando por idade
```

```
ordenado = sorted(peessoas, key=lambda p: p["idade"])  
print("Ordenado por idade:")  
print(ordenado)
```

```
#Ordenando por ordem alfabética:
```

```
ordenado = sorted(peessoas, key=lambda p: p["nome"])  
print("\nOrdenando por ordem alfabética:")  
print(ordenado)
```

```
#Ordenando por idade decrescente
```

```
ordenado = sorted(peessoas, key=lambda p: p["idade"], reverse=True)  
print("Ordenado por idade reversa:")  
print(ordenado)
```

Funções em Python

Função `enumerate()`.

- `enumerate()` Adiciona índices a um objeto iterável.

Ex. recebe uma lista e retorna um objeto adicionando os índices.

Sintaxe:

```
enumerate(iterável, start=1)
```

iterável: objeto a ser percorrido.

start: opcional - início da contagem, se não informado, inicia em 0.

Funções em Python

Função enumerate().

- `enumerate()`:

```
frutas = ["maçã", "banana",  
"laranja"]
```

```
y = enumerate(frutas)  
print(frutas)  
print(y)  
print(list(y))
```

```
#Imprimir uma lista de itens numerados  
#Sem enumerate()
```

```
frutas = ["maçã", "banana", "laranja"]  
print(frutas)
```

```
for i in range(len(frutas)):  
    print(f"{i + 1}. {frutas[i]}")
```

```
print()
```

```
#Com enumerate
```

```
frutas = ["maçã", "banana", "laranja"]  
print(list(enumerate(frutas, start=1)))
```

```
for i, fruta in enumerate(frutas, start=1):  
    print(f"{i}. {fruta}")
```

Funções em Python

- A palavra-chave `import` é usada para trazer código de outro módulo (arquivo) para o programa atual. Com isso, você pode usar funções, variáveis, classes ou qualquer outra coisa que esteja definida em outro arquivo ou biblioteca;
- Um módulo em Python é simplesmente um arquivo `.py` que contém código (funções, variáveis, etc.).

```
#Aqui, você está importando o módulo math e usando a função sqrt() (raiz quadrada).
```

```
import math
```

```
print(math.sqrt(25)) # Saída: 5.0
```


Funções em Python

Sintaxe

Explicação

```
import math
```

Importa o módulo todo (acessa com `math.funcao`)

```
from math import sqrt
```

Importa só a função `sqrt`

```
from math import *
```

Importa **tudo** do módulo (não recomendado para programas grandes)

```
import math as m
```

Importa com um nome abreviado

Funções em Python

```
import math
```

```
print(math.sqrt(25))    # Saída: 5.0 ( o modulo vem em forma de objeto e não gera  
                        #conflito com código local )
```

```
from math import sqrt
```

```
print(sqrt(25))    # Saída: 5.0 ( importa somente o que for especificado )
```

```
from math import *
```

```
print(sqrt(25))    # Saída: 5.0 ( evitar!, importa TODOS os nomes do módulo direto  
para seu  
                        #escopo, pode gerar conflito com código local )
```

```
import math as mt
```

```
print(mt.sqrt(25))    # Saída: 5.0 ( o modulo vem em forma de objeto )
```

Funções em Python

- Embora Python não exija uma função `main()` como em outras linguagens (como C, C++ ou Java), é comum usá-la para organizar o código, especialmente em programas maiores;
- Você pode definir uma função chamada `main()` e chamá-la usando o seguinte padrão:

```
def main():  
    print("Este é o início do programa.")  
  
# Isso garante que o main só será executado se o script for rodado diretamente  
if __name__ == "__main__":  
    main()
```

Funções em Python

- `__name__` é uma variável interna que o Python cria automaticamente em cada arquivo que é executado ou importado;
- Se o arquivo está sendo executado diretamente, `__name__` será igual a `"__main__"`;
- Se o arquivo está sendo importado como módulo, `__name__` será igual ao nome do arquivo (sem .py).

minhas_funcoes.py



import



programa_principal.py



```
def checa_nome():  
    print(f"__name__: {__name__}")  
  
def saudacao():  
    print("Olá!")  
  
checa_nome()  
#__name__ = __main__
```

```
import minhas_funcoes  
  
#__name__ = minhas_funcoes
```

Funções em Python

- `if __name__ == "__main__":` : essa linha é uma verificação condicional. Ela pergunta: "Este arquivo está sendo executado diretamente do arquivo-fonte?";
- Se sim, o Python executa o que está dentro do `if`;
- Se não (ou seja, o arquivo foi apenas importado em outro), o código dentro do `if` não será executado automaticamente, quando o arquivo for executado.

minhas_funcoes.py



import



programa_principal.py



```
def checa_nome():  
    print(f"__name__: {__name__}")  
  
def saudacao():  
    print("Olá!")  
  
if __name__ == "__main__":  
    checa_nome()  
    saudacao()  
  
#saída:  
#__name__: __main__  
#Olá!
```

```
import minhas_funcoes  
  
#sem saída
```

Funções em Python

- Sobre a função `main()`, apesar de não ser obrigatória, é uma boa prática organizar as chamadas de funções dentro da função principal `main()`.

```
def main():  
    print("Este é o início do programa.")  
  
# Isso garante que o main só será executado se o script for rodado diretamente  
if __name__ == "__main__":  
    main()
```


minhas_funcoes.py



import



programa_principal.py



```
def checa_nome():  
    print(f"__name__: {__name__}")  
  
def saudacao():  
    print("Olá!")  
  
def main():  
    checa_nome()  
    saudacao()  
  
if __name__ == "__main__":  
    main()  
  
#saída:  
# __name__: __main__  
#olá!
```

```
import funcoes  
  
funcoes.saudacao()  
  
#saída:  
#olá!  
  
#Apenas o que realmente for  
#executado no arquivo
```

Funções em Python

```
def cadastrar_alunos():
    alunos = []
    for i in range(3):
        nome = input(f"Digite o nome do aluno {i+1}: ")
        nota1 = float(input("Nota 1: "))
        nota2 = float(input("Nota 2: "))
        aluno = {
            "nome": nome,
            "notas": [nota1, nota2]
        }
        alunos.append(aluno)
    return alunos

def mostrar_medias(alunos):
    print("\nMÉDIAS DOS ALUNOS:")
    for aluno in alunos:
        media = sum(aluno["notas"]) / len(aluno["notas"])
        print(f"{aluno['nome']}: {media:.2f}")

def main():
    turma = cadastrar_alunos()
    mostrar_medias(turma)

if __name__ == "__main__":
    main()
```

Exercício

- Você vai criar um programa que gerencia uma lista de tarefas. O programa deve:
 1. Permitir que o usuário adicione tarefas. Cada tarefa deve ter: Descrição (string), Prioridade (inteiro de 1 a 5, onde 1 é a prioridade mais alta) e Status (inicialmente “pendente”);
 2. Armazenar cada tarefa como um dicionário com as chaves "descrição", "prioridade" e "status";
 3. Manter todas as tarefas numa lista;
 4. Criar funções para: Adicionar tarefas, Mostrar todas as tarefas ordenadas por prioridade (da maior para a menor) e Marcar uma tarefa como concluída (alterar o status para “concluída”);
 5. Criar uma função main() que permita ao usuário escolher opções no menu para: Adicionar tarefa, Listar tarefas, Marcar tarefa como concluída e Sair do programa.

FIM

